Cross-Chain Governance

Pre-Requisite Reading

- Nomad: Optimistic Interchain Communication

Summary

Purpose

This document describes **a governable system for executing permissioned actions across chains**.

We aim to clearly describe

- **what** contracts comprise the system for calling permissioned functions across chains
- **which** functions will be delegated to this system at launch, and
- (directionally) **who** will have permission to call these functions at launch and in the future

Out of Scope

This document does NOT describe a system for **how** governance actions will be proposed, voted on, and/or approved before being executed.

It does not describe how contract upgrades will be written, reviewed, verified.

Overview

We define a role, `governor`, with the power to perform permissioned actions across chains. In order to empower the `governor`, we deploy a cross-chain application comprised of a `GovernanceRouter` contract on each chain.

Each `GovernanceRouter` can be delegated control over an arbitrary set of permissioned functions on its local chain. The only way to access the permissioned functionality is to call the function via the `GovernanceRouter` contract.

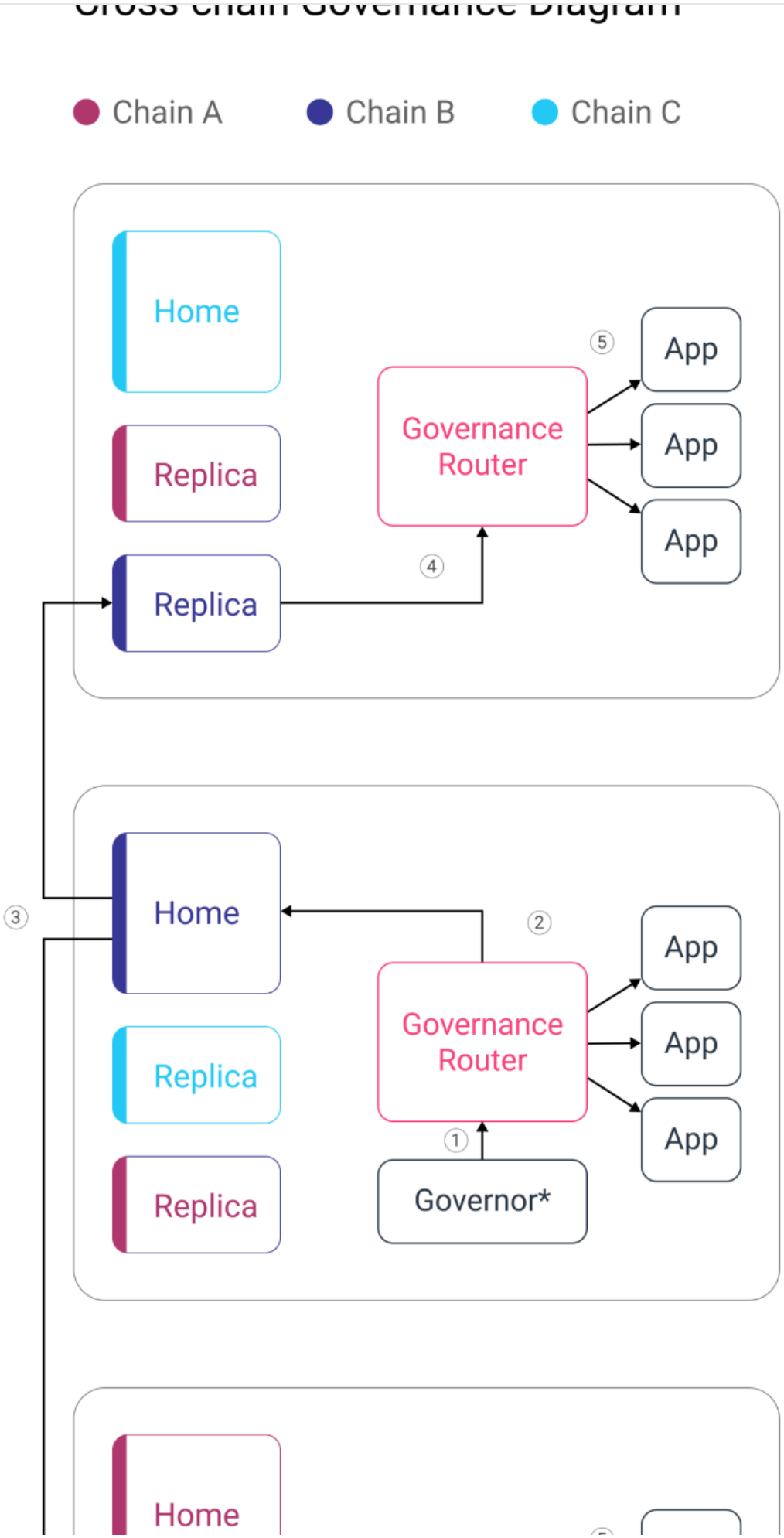
the same chain), or it may send it messages remotely via Nomad. Because of its exclusive power over the `GovernanceRouter` contracts, the `governor` has exclusive rights to perform **all** of the permissioned roles that are delegated to the `GovernanceRouter` on each chain.

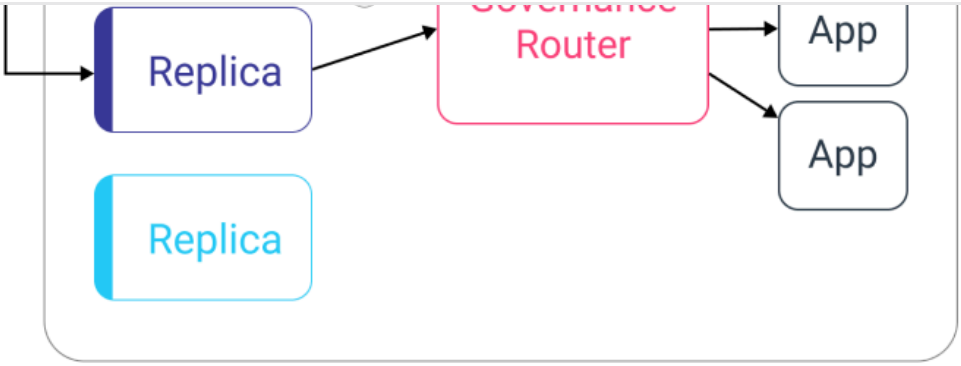
The system receives orders from the `governor` and carries out their effects across chains; it is agnostic to how the `governor` chooses to operate. This maintains flexibility to design the governance proposal process in the future.

At launch, the core functionality that will be delegated to the `GovernanceRouter` on each chain is the power to upgrade the implementation of the `Home` and `Replica` contracts. This way, the `governor` will have the power to conduct upgrades of the Nomad system on every chain. More details on the upgradability system can be found here.

At launch, the `governor` will be a multisig of trusted team and community members. In the near future, the `governor` role will most likely be transferred to a more fully-featured set of contracts capable of accepting proposals, tallying votes, and executing successful proposals.

Message Flow Diagram


Chain A
Chain B
Chain C
Home
Replica
Replica
Governance Router
App
App
App
5
4
3
Home
Replica
Replica
Governance Router
Governor*
App
App
App
2
1
Home



1. `governor` sends message to its local `GovernanceRouter`
2. `GovernanceRouter` dispatches the message...
    1. if the recipient is local, to the recipient directly (→ process finished)
    2. if the recipient is remote, via Nomad to the local Home contract (→ continue to 3)
3. Message is relayed from local `Home` to remote `Replica` via Nomad
4. `Replica` dispatches message to the remote `GovernanceRouter`
5. `GovernanceRouter` dispatched the message directly to the local recipient

Note on message recipient:

- the recipient may be a `Replica` or `Home` contract
- it may be an `UpgradeBeacon` that controls the implementation of `Replica` or `Home`
- it may be any other app

For simplicity & clarity to show the message flow, this diagram represents the recipient as a generic "App"

Specification

Glossary of Terms

- **xApp** - Cross-Chain Application
- **role** —
    - an address stored in a smart contract's state that specifies an entity with special permissions on the contract
    - permission to call certain functions is usually implemented using a function modifier that requires that the caller of the function is one of the roles with permission to call it; all contract calls sent from callers that do not have valid permission will revert

- **permissioned function** —
  - any smart contract function that restricts callers of the function to a certain role or roles
  - *example*: functions using the `onlyOwner` modifier on Ownable contracts
- **permissioned call** — a call to a **permissioned function**
- **governor chain** —
  - the chain on which the `governor` is deployed
  - the chain whose `GovernanceRouter` is also the special `GovernorRouter` which can *send* messages; all `GovernanceRouters` on other chains can only *receive* governance messages

On-Chain Components

GovernanceRouter

- xApp designed to perform permissioned roles on core Nomad contracts on all chains
- State Variables
- **governor** state variable
  - if the `governor` is local, `governor` will be set to the EVM address of the `governor`
  - if the `governor` is remote, `governor` will be `address(0)`
- **governorDomain** state variable
  - the Nomad domain of the **governor chain**
  - stored as a state variable on all `GovernanceRouters`; should be the same on all `GovernanceRouters`; always non-zero
    - if the `governor` is local, `governorDomain` is equal to the `originDomain` of the local `Home` contract
    - if the `governor` is remote, `governorDomain` is equal to the `originDomain` of the remote `Home` contract
  - equal to the `originDomain` of the local `Home` contract on the chain of the `GovernorRouter`
  - used by all `GovernanceRouters` to determine whether an incoming Nomad message was sent from the `GovernorRouter`
    - if the message is from the `GovernorRouter`, the `GovernanceRouter` will handle the incoming message
    - if not, it will revert
- **routers** state variable
  - a mapping of domain → address of the remote `GovernanceRouter` on every other chain
- **domains** state variable
  - an array of all domains that are registered in `routers`

registered)

- **GovernorRouter**
  - the special `GovernanceRouter` that has *permission to send* governance messages to all other `GovernanceRouters`
  - the `GovernanceRouter` on the **governor chain**

Governor

- via the `GovernanceRouter` system, it has the unique ability to call permissioned functions on **any contract** on **any chain** that transfers permission to the local `GovernanceRouter`
- the **role** with permission to send messages to the `GovernorRouter`
  - the `GovernorRouter` has exclusive permission to send messages via Nomad to all other `GovernanceRouters`
  - the `GovernanceRouters` can have arbitrary permissions delegated to them by any contract on their local chain
  - therefore, the `governor` is the entity with the power to call any **permissioned function** delegated to any `GovernanceRouter` on any chain
- there is only one `governor` throughout the Nomad system; it can be deployed on any chain
- the `governor` role can always be transferred to another contract, on the same chain **or** a different remote chain
- stored as a state variable on `GovernanceRouters` ; set to zero on all `GovernanceRouters` except on the **governor chain**
- **Any contract** on **any chain** that wishes for this governance system to have discretion to call a set of its functions can create a role & a function modifier giving exclusive permission to that role to call the function(s) (similar pattern to Ownable). The contract must then set the local `GovernanceRouter` to the permissioned role, which — by extension — gives the `governor` exclusive permission to call those functions (regardless of whether the `governor` is remote or local)

Failure States

If there is fraud on the Nomad `Home` contract on the **governor chain**, this is currently a "catastrophic failure state" — no further governance actions can be rolled out to remote chains; we must create a plan to recover the system in this case.

Executing (Arbitrary) Calls

1. **for each chain**, the `governor` constructs the array of `(to, data)` calls to the permissioned functions on the contracts that will perform the upgrades on that chain
2. the `governor` sends a transaction to the `GovernanceRouter.callRemote` function on its local the , passing in the `domain` of the remote chain and the array of `(to, data)` calls of transactions to execute on that chain
3. the local `GovernanceRouter` constructs an Nomad-compatible message from the array of calls, addresses the message to the remote `GovernanceRouter` , and sends the message to the local `Home` contract
4. the message is relayed from the local `Home` to the remote `Replica` contract on the specified `domain`
5. the `Replica` dispatches the message to the specified recipient, which is the local `GovernanceRouter`
6. the `GovernanceRouter` parses the message to decode the array of `(to, data)` calls
7. the `GovernanceRouter` uses low-level call to execute each of the transactions in the array within the local chain

Transferring Governor

Possible State Transitions

1. called by the local owner to transfer ownership to another local owner (`domain` does not change, `owner` changes to a new `bytes32` address)
2. called by the local owner to transfer ownership to a remote owner (`domain` changes to the remote, `owner` changes from a non-zero `bytes32` to `bytes32(0)`)
3. called by a remote owner to transfer ownership to a local owner (`domain` changes to the local domain, `owner` changes from `bytes32(0)` to a non-zero `bytes32`)
4. called by a remote owner to transfer ownership to another remote owner (`domain` changes to the new remote owner, `owner` remains `bytes32(0)`)

Enrolling a Router

- used when a new chain is added to Nomad after we've already set up the system and transferred governorship
- add a new domain → address mapping to the `routers` mapping on every other `GovernanceRouter`

Permissioned Roles

At launch, the `GovernanceRouter` system **will have the following permissions**:

1. upgrade the implementation of `Home` (via `UpgradeBeacon` pattern)
2. upgrade the implementation of all `Replicas` (via 1-to-N `UpgradeBeacon` pattern)
3. upgrade the implementation of itself (via `UpgradeBeacon` pattern)

The `GovernanceRouter` **will NOT have permission** to:

- un-enroll a `Replica` from the `UsingNomad` contract, which will require a specialized role that can act quickly

Governor

The flexibility of this system will support a move to progressive decentralization.

Initially, the `governor` will most likely be a multisig controlled by trusted team and community members

Later, the `governor` role will most likely be transferred to a decentralized governance contract